1 E2: Transformer (MSA và tokenization)

1.1 Mục tiêu

Bài này hiện thực multi-head self-attention (MSA) hai lần trên cùng tập MNIST và cùng split đã dùng ở E1: 50 000 ảnh train, 10 000 ảnh validation (stratify, seed 42) và 10 000 ảnh test. Bản thứ nhất viết từ `Linear`, `einsum` và softmax. Bản thứ hai bọc `nn.MultiheadAttention` để đối chiếu. Cả hai đi vào một encoder nông và một head phân loại. Ba cách lấy token bắt buộc là patch, hàng và CNN stem; thêm patch 7×7 và patch chồng lấn.

1.2 Mô hình

Ảnh $x \in \mathbb{R}^{B\times 1\times 28\times 28}$ được tokenizer đổi thành chuỗi N token, mỗi token thuộc $\mathbb{R}^D$ với $D = 64$. Một token CLS được ghép vào đầu, rồi cộng positional embedding học được kích thước $(1, N+1, D)$, khởi tạo `trunc_normal_` với độ lệch chuẩn 0.02 (`TransformerClassifier` trong `models/e2/transformer.py`).

Mỗi `EncoderBlock` là pre-norm. Gọi $h = 4$ head, $d_k = D/h = 16$:

$$\begin{aligned} Q, K, V &= \mathrm{split}(W_{\text{in}}\,\mathrm{LN}(X)), \\ A &= \mathrm{Dropout}\left(\mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)\right), \\ \mathrm{MSA}(X) &= W_{\text{out}}\,\mathrm{concat}(AV), \\ X &\leftarrow X + \mathrm{Dropout}(\mathrm{MSA}(\mathrm{LN}(X))), \\ X &\leftarrow X + \mathrm{FFN}(\mathrm{LN}(X)). \end{aligned} \tag{1}$$

FFN là `Linear` → GELU → Dropout → `Linear`, chiều ẩn $2D$. Trong `ScratchMSA` (`models/e2/attention.py`), tích $QK^\top$ và AV là hai phép `einsum`: `'bhnd,bhmd->bhnm'` và `'bhnm,bhmd->bhnd'`. Trọng số W_{in} xếp $[W_Q; W_K; W_V]$, cùng layout với `in_proj_weight` của PyTorch, khởi tạo Xavier và bias bằng 0. `TorchMSA` gọi `nn.MultiheadAttention(batch_first=True)`. Khi không cần trả trọng số attention, PyTorch dùng kernel SDPA; đó là một lý do thời gian hai bản có thể khác nhau dù cùng kiến trúc còn lại.

Sau 4 block và một `LayerNorm`, lớp `Linear` $D \to 10$ đọc token CLS. `forward` trả logits cho `CrossEntropyLoss`. `return_attn=True` trả thêm attention từng layer, dùng khi vẽ bản đồ attention.

Ba tokenizer trong `models/e2/tokenizers.py`:

Tokenizer	Cách lấy token	N	Ghi chú
Patch 4×4	`Conv2d` kernel = stride = 4	49	ô không chồng
Hàng	`Linear` trên mỗi hàng $1 \cdot 28$	28	cùng cách nhìn ảnh với E3
CNN stem	hai lần Conv–BN–ReLU–Pool, rồi Conv 1×1	49	bias cục bộ

Biến thể thêm, chỉ với MSA tự viết: patch 7×7 ($N = 16$), và patch chồng lấn kernel 8 stride 4 ($N = 36$). Không lấy token theo kênh màu vì MNIST có $C = 1$.

Mọi run dùng AdamW, learning rate 10^{-3}, batch 128. Weight decay 0.05 chỉ áp lên trọng số của `Linear` và `Conv2d`; bias, tham số của LayerNorm và BatchNorm, token CLS và positional embedding không bị decay, theo cách train ViT thông thường (`TransformerClassifier.param_groups`). Train tối đa 100 epoch. `ReduceLROnPlateau` giảm learning rate $\times 0.1$ sau 10 epoch loss validation không giảm; early stopping dừng sau 20 epoch, giống E1, để mô hình còn 10 epoch học với learning rate đã giảm. `EpochTimer` ghi `training/epoch_time` là thời gian các bước train, không cộng thời gian validation.

1.3 Kiểm tra MSA

`scripts/e2/check_msa.py` copy trọng số từ `nn.MultiheadAttention` sang `ScratchMSA`, chạy ở `eval` để tắt dropout, rồi so output, attention từng head và gradient theo input cùng theo trọng số. Output và attention ở float32 phải lệch dưới 10^{-5}. Gradient float32 được chấp nhận dưới 10^{-4}, vì phép cộng ngược của PyTorch không cùng thứ tự với `einsum`. Kết quả nằm ở `docs/assets/e2/msa_check.csv`. Cùng script kiểm tra shape: tokenizer trả $(2, N, 64)$, classifier trả $(2, 10)$.

1.4 Huấn luyện và số liệu

Sáu run bắt buộc là {patch, hàng, CNN stem} × {scratch, torch}. So sánh (i) đọc theo hàng: cùng tokenizer, đổi bản MSA. So sánh (ii) đọc theo cột: cùng bản MSA, đổi tokenizer, về accuracy, số tham số và thời gian mỗi epoch.

`scripts/e2/analyze.py` đánh giá checkpoint của mỗi run trên tập test và ghép với log TensorBoard được tạo cùng lúc với checkpoint đó. Validation accuracy được đọc tại epoch có loss validation thấp nhất, cũng là epoch mà `BestCheckpoint` giữ lại, nên Val acc và Test acc trong cùng một dòng thuộc về cùng một mô hình. Script ghi `results_e2.csv` và `results_e2.tex` (Bảng 1), đường loss và accuracy `curves_<tokenizer>.png` (train nét đứt, validation nét liền, mỗi bản MSA một màu), `acc_vs_time.png`, và `confusion_best.png` cho run có validation accuracy cao nhất. Attention của CLS ở layer cuối, từng head, là `attn_<run>.png`.

Table 1: Kết quả E2 trên MNIST. Val acc lấy tại epoch có loss validation thấp nhất (Epoch); Test acc của checkpoint ở epoch đó; Time/epoch chỉ tính các bước train.

Tokenizer	MSA	Params	Epoch	Time/epoch (s)	Val acc (%)	Test acc (%)	Ảnh/s
patch	scratch	139 018	34	12.05	98.63	98.75	4902
patch	torch	139 018	82	16.46	98.70	98.81	4960
row	scratch	138 442	47	16.28	98.98	99.19	4907
row	torch	138 442	47	17.09	98.96	98.95	4955
cnnstem	scratch	161 098	30	17.97	99.32	99.36	4919
cnnstem	torch	161 098	29	16.59	99.23	99.35	4959
patch7	scratch	139 018	25	11.27	98.54	98.67	4904
patch_overlap	scratch	141 258	47	16.14	98.97	99.11	4905

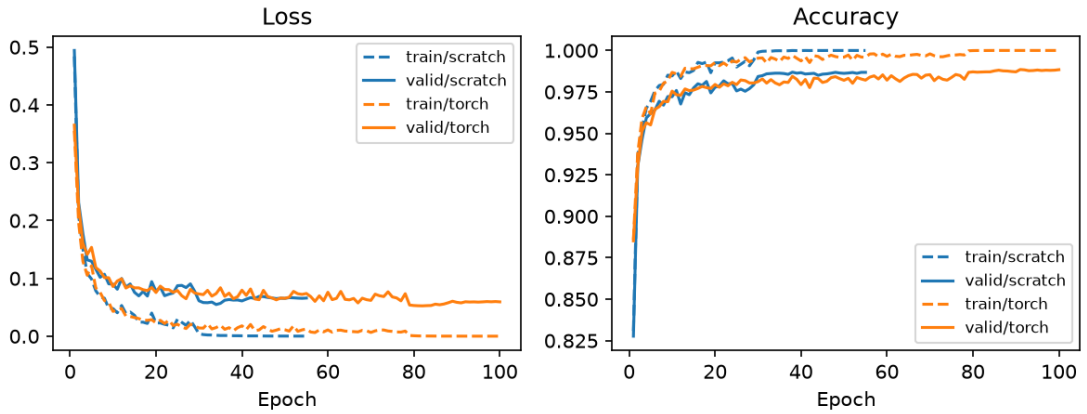


Figure 1: Loss và accuracy, tokenizer patch 4 × 4.

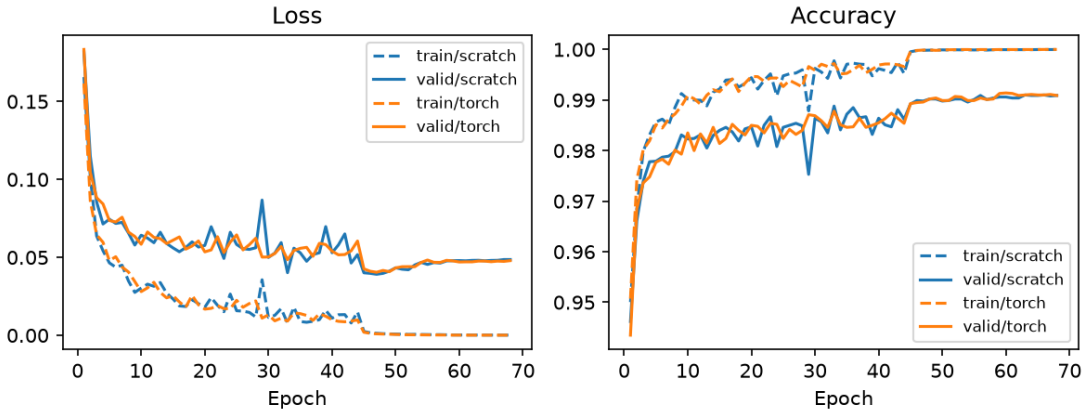


Figure 2: Loss và accuracy, tokenizer theo hàng.

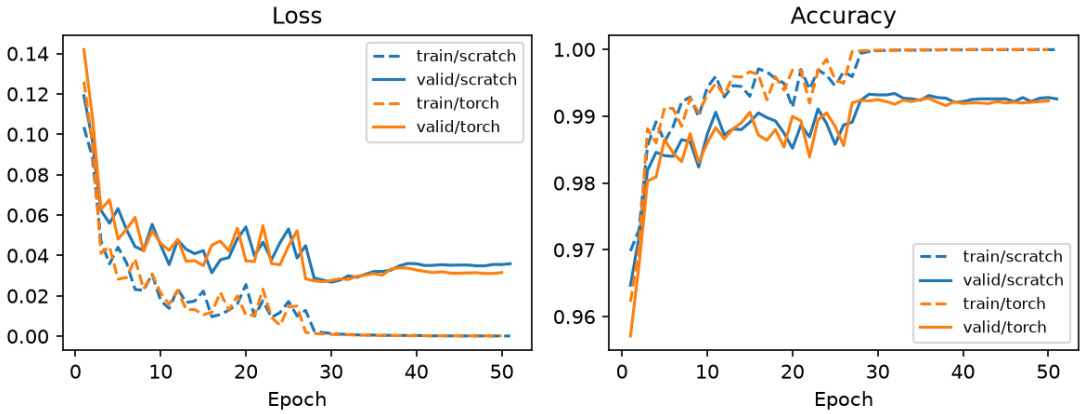


Figure 3: Loss và accuracy, tokenizer CNN stem.

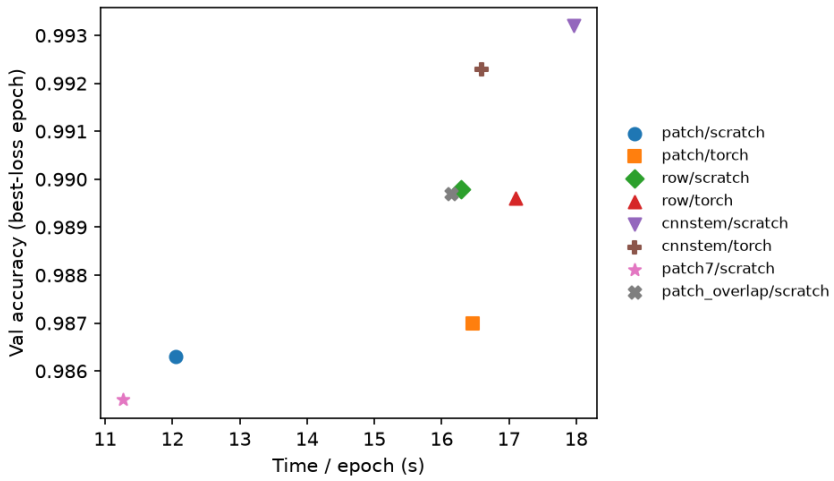


Figure 4: Validation accuracy theo thời gian train mỗi epoch.

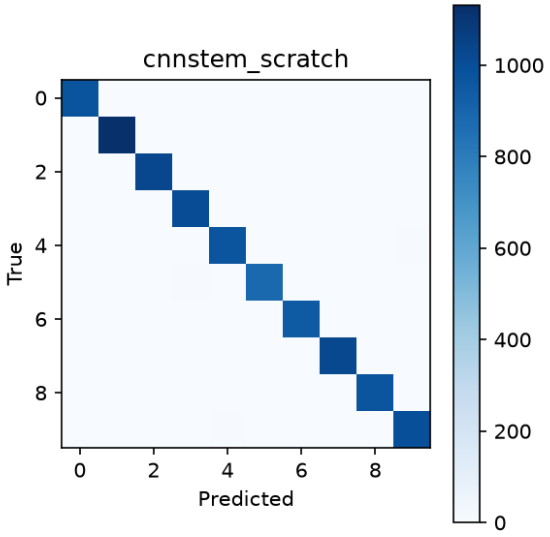


Figure 5: Confusion matrix trên tập test của run có validation accuracy cao nhất.

1.5 Thảo luận

Hai bản MSA chia sẻ tokenizer, LayerNorm, FFN, CLS và head. Mỗi cấu hình chỉ chạy một seed (42), nên mọi chênh lệch dưới khoảng 0.3 điểm phần trăm trong phần này chưa đủ để kết luận; muốn kiểm tra, copy config patch và đổi `seed` thành 43 và 44.

Scratch so với torch (Bảng 1). Test accuracy của hai bản gần nhau ở cả ba tokenizer: patch 98.75% so với 98.81%, hàng 99.19% so với 98.95%, CNN stem 99.36% so với 99.35%. Chênh lệch lớn nhất là 0.24 điểm (hàng), và hướng chênh không nhất quán (torch cao hơn ở patch, thấp hơn ở hàng), nên phù hợp với nhiễu tối ưu hơn là một khác biệt hệ thống. Kết hợp với kiểm tra số học ở `msa_check.csv`, bản MSA tự viết cho kết quả tương đương `nn.MultiheadAttention`. Cặp patch dừng ở các epoch rất khác nhau (34 so với 82): đây là hệ quả của early stopping và `ReduceLROnPlateau` trên một đường loss validation phẳng, không nên đọc thành một bản hội tụ nhanh hơn bản kia.

Số tham số của cặp scratch/torch là như nhau: patch 4×4 và patch 7×7 đều 139 018 (kernel lớn hơn được bù bởi positional embedding ngắn hơn), hàng 138 442, patch chồng lấn 141 258 và CNN stem 161 098.

Số token quyết định giá attention. Patch 7×7 có 16 token, rẻ hơn patch 4×4 (49 token) nhưng mỗi token phải ôm một vùng lớn hơn. CNN stem cũng ra 49 token như patch 4×4, nhưng stem đã trộn lân cận trước khi attention nhìn toàn cục. Hàng chỉ có 28 token và mỗi token là cả một hàng, nên attention mô hình hóa quan hệ giữa các hàng, không phải giữa các ô.

Tokenizer so với tokenizer. Với MSA tự viết, test accuracy xếp từ cao xuống thấp là: CNN stem 99.36%, hàng 99.19%, patch chồng lấn 99.11%, patch 4×4 98.75%, patch 7×7 98.67%. CNN stem dẫn đầu với số tham số lớn nhất (161 098, hơn các mô hình còn lại khoảng 16%), nên chưa tách được phần đóng góp của bias cục bộ khỏi phần đóng góp của dung lượng lớn hơn. Hai tokenizer dựa trên patch không chồng lấn là yếu nhất, và patch 7×7 (16 token) thấp hơn patch 4×4 (49 token), phù hợp với việc mỗi token ôm vùng lớn hơn nhưng có ít token hơn để attention trộn. Patch chồng lấn (kernel 8, stride 4, 36 token) cao hơn patch 4×4 khoảng 0.36 điểm với cùng bậc số tham số (141 258 so với 139 018); các vùng chồng nhau có thể giúp token biên nhìn thấy nét liền mạch, nhưng với một seed đây mới là giả thuyết.

Thời gian. Time/epoch dao động từ 11.27 s (patch 7×7) đến 17.97 s (CNN stem), và patch 7×7 nhanh nhất đúng như số token ít nhất. Tuy vậy các run được chạy song song trên các container GPU riêng, nên chênh lệch nhỏ (ví dụ patch scratch 12.05 s so với torch 16.46 s) không

đủ để kết luận bản nào nhanh hơn; cột Ảnh/s (4700–4900 cho mọi mô hình) cho thấy tốc độ suy luận gần như không phân biệt được giữa các cấu hình ở quy mô này.

Tất cả tám mô hình đạt trên 98.6% test accuracy với chưa tới 162 000 tham số, nên MNIST khó phân biệt các tokenizer; khoảng cách lớn nhất giữa mô hình tốt nhất và kém nhất chỉ 0.69 điểm. Muốn so sánh chắc chắn hơn cần nhiều seed hoặc một tập khó hơn.

1.6 Sử dụng công cụ AI

Mã trong `models/e2`, `configs/e2`, `scripts/e2`, `common/handlers/timer.py` và chương này được soạn với sự hỗ trợ của công cụ AI trong Cursor, theo đặc tả E2 của nhóm. Công cụ chỉ dùng để viết khung mã, config và lời giải thích. Số liệu thí nghiệm chỉ phát sinh khi chạy lệnh train và evaluate trong repo. Người nộp bài cần đọc và giải thích được từng file trên trước khi bảo vệ.